Overview

The bus booking system back end users layered implicit exception handling rather than rapping every call in a try catch the architecture let us service layer code through typed .Net and we can map them to HTTP status codes

layer by layer breakdown

1. global exception handler middleware

   so here any unhandled exception bubbling up from the controller or services so it used as a sharp pattern matching switch expression to map .Net exception types to HTTP status codes

Status code mapping

```
ctx.Response.StatusCode = exception switch
{
    UnauthorizedAccessException => 401,
    KeyNotFoundException        => 404,
    ArgumentException           => 400,
    _                           => 500
};
```

Status code

```
{
  "error": "Trip not found",
  "statusCode": 404
}
```

Returns a consistent Json body

2. Service Layer

   The services do not use try catch themselves they throw specific exception types when business rules are violated and the global handler catches them

Business logic Errors

```
// Invalid role → 400
if (!validRoles.Contains(dto.Role.ToLower()))
    throw new ArgumentException("Invalid role. Must be 'customer' or 'operator'");
```

3. Fire and Forget Email (Isolated try/catch)

   Sending email is non critical side effect the system deliberately wraps all email calls in their own try catch so that an email failure does not rollback or fail to primary operation like it does not go up till the global exception handling it will handle in the same area.

Email Exception Handling in Isolated Environment

```
// Send cancellation email (fire and forget)
try
{
    await _emailService.SendCancellationEmailToAsync(booking, recipientEmail, recipientName);
}
catch (Exception ex)
{
    _logger.LogError(ex, "Failed to send cancellation email for booking {BookingRef}", booking.BookingRef);
}

```

4. Controller level inline guards hybrid approach

   Here the admin controller having these separate exceptional handling like So what are the things it having it will store the exceptions in the log format so these type of things does not accidentally shown to the end user

Logging of the Admin Controller Exceptions

```
// AdminController.cs — BlockOperator
if (string.IsNullOrEmpty(adminEmail))
    return Unauthorized(new { error = "Administrator identity not found in session." });

if (admin == null)
    return Unauthorized(new { error = "Administrator account not found." });

if (!BCrypt.Net.BCrypt.Verify(dto.Password, admin.PasswordHash))
{
    _logger.LogWarning("Failed block operator attempt: Invalid password for admin {Email}", adminEmail);
    return StatusCode(403, new { error = "Invalid administrator password. Access denied." });
}

```

5. User status middleware infrastructure level code

   here a secondary middle Ware intercepts every authenticator request and blocks it at the HTTP pipeline level if the user account is inactive

User Status Middleware

```
// Middleware/UserStatusMiddleware.cs
if (user == null || !user.IsActive)
{
    context.Response.StatusCode = StatusCodes.Status403Forbidden;
    await context.Response.WriteAsJsonAsync(new { error = "Account is inactive
or blocked. Access denied." });
    return; // Short-circuit — does not proceed to controllers
}

```